

Backend Blueprint v1 — La Sentadita Hub

1. OBJETIVO

Construir un backend que:

- aplique el dominio ya congelado
- centralice autorización y reglas de negocio
- use Supabase/PostgreSQL como capa de persistencia
- no dependa únicamente de RLS para lógica crítica
- pueda implementarse por fases con apoyo de IA

Regla base:

El backend es la capa de verdad del negocio; la base de datos guarda y protege, pero no decide sola.

2. PRINCIPIOS DE ARQUITECTURA

P1. Backend modular por dominio

Módulos:

authz, people, employment, scheduling, tasks, procedures, shift-swaps, incidents, documents, notifications, audit

P2. Casos de uso explícitos

Cada operación importante debe tener un caso de uso claro.

Ejemplos:

PublishSchedule

ApproveProcedure

RequestShiftSwap

ApplyApprovedShiftSwap

ReassignTask

P3. Autorización centralizada

Evitar repetir lógica de permisos en todo el sistema.

Un módulo authz responde si una acción está permitida.

P4. Side effects orquestados

Las acciones que afectan varios módulos deben coordinarse desde el caso de uso.

P5. Auditoría obligatoria

Toda acción crítica debe registrar auditoría.

P6. Frontend tonto, backend inteligente

El frontend muestra y solicita; el backend decide y valida.

3. STACK RECOMENDADO

Runtime:

Next.js App Router

TypeScript

Server Actions o Route Handlers

Supabase client (service role en backend)

Base de datos:

PostgreSQL / Supabase

Validación:

Zod

4. ESTRUCTURA DE CARPETAS

```
src/  
  modules/  
    authz/  
    people/  
    employment/  
    scheduling/  
    tasks/  
    procedures/  
    shift-swaps/  
    incidents/  
    documents/  
    notifications/  
    audit/  
  shared/  
    db/  
    types/  
    errors/
```

utils/
validation/

5. CAPAS

domain/
Reglas puras del negocio.

application/
Casos de uso del sistema.

infrastructure/
Acceso técnico a base de datos, storage y proveedores.

6. REQUEST CONTEXT

Toda operación crítica debe usar un RequestContext:

```
type RequestContext = {  
  personId: string  
  systemRole: SystemRole  
  activeScopeType: ScopeType | null  
  activeScopeId: string | null  
  traceId: string  
  now: Date  
}
```

7. MÓDULO AUTHZ

Responsable de determinar permisos.

API ejemplo:
can(ctx, action, resource)
assertCan(ctx, action, resource)

8. MÓDULOS DEL SISTEMA

People

- CreatePerson
- UpdatePersonIdentity
- ArchivePerson
- GetPersonProfile

Employment

- CreateEmploymentRelationship
- UpdateEmploymentOperationalFields
- UpdateEmploymentContractualFields
- TerminateEmploymentRelationship

Scheduling

- GetScheduleDraft
- UpdateScheduleEntry
- AcquireScheduleLock
- PublishSchedule
- RepublishSchedule

Tasks

- CreateTaskTemplate
- CreateManualTask
- CompleteTask
- ReassignTask

Procedures

- CreateProcedure
- ApproveProcedure
- RejectProcedure
- DeriveAbsence

Shift Swaps

- RequestShiftSwap
- AcceptShiftSwapPeer
- ApproveShiftSwap
- RejectShiftSwap

Incidents

- CreateIncident
- AssignIncident
- ChangeIncidentStatus

Documents

- UploadDocument

- ViewDocument
- ArchiveDocument

Notifications

- CreateNotification
- SendPushNotification

Audit

- WriteAuditLog
- ListAuditLogs

9. PATRÓN DE CASO DE USO

1. construir RequestContext
2. validar input
3. cargar entidades
4. assertCan(...)
5. validar invariantes
6. ejecutar operación
7. aplicar side effects
8. registrar auditoría
9. emitir notificaciones
10. devolver resultado

10. ORDEN DE IMPLEMENTACIÓN

Fase 1

RequestContext
authz
audit
repositorios base

Fase 2

people
employment

Fase 3

scheduling
tasks

Fase 4

procedures
shift-swaps

Fase 5

incidents
documents

Fase 6

notifications

CONCLUSIÓN

Este blueprint establece una arquitectura modular, auditable y segura para el backend de La Sentadita HU, permitiendo implementar el sistema de forma incremental manteniendo consistencia con el modelo de dominio.